

Task 01 — Complete puzzle backend

Owner: Eyad. Branch: `member-eyad`. Reviewer: Majed. Project deadline: 3 October 2026, 23:59 Asia/Damascus.

Context

Build all four backend APIs. Continue your existing Express setup and startup configuration.

Read [docs/phase-1-agreement/apis-handoff.pdf](#), [docs/phase-1-agreement/puzzles-reference.pdf](#), and [docs/team/requirements.md](#). Use the existing three puzzles in [server/data.js](#). This task defines the response fields and progress rules needed to implement the API agreement.

Goal

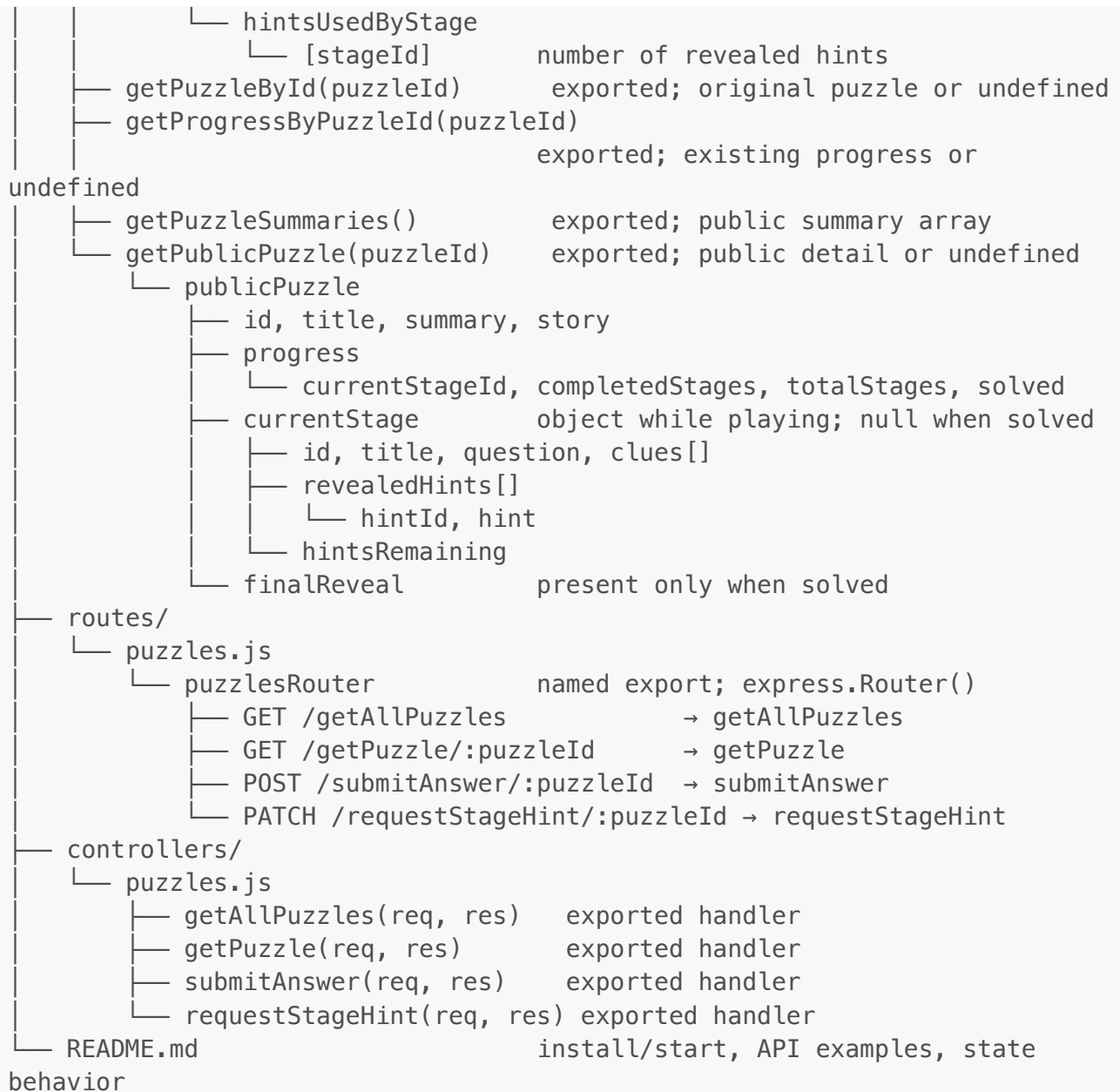
The frontend can list puzzles, read the current stage, submit answers, request hints, and receive the final reveal through real APIs.

What to build

Content tree

Create or update these files. Use these names for the listed functions and objects. Other temporary variables can use clear names.

```
server/
├── package.json
│   ├── type: "module"
│   └── scripts.start: "node index.js"
├── package-lock.json
├── index.js
│   ├── app                                Express application
│   ├── PORT                              process.env.PORT || 3001
│   ├── puzzlesRouter                      imported Router
│   └── handleJsonParseError(err, req, res, next)
├── data.js
│   ├── puzzlesData                        named export; original puzzle content
│   │   └── puzzles[]
│   │       ├── id, title, summary, story
│   │       ├── stages[]
│   │       │   ├── id, title, question, clues[], answers[], hints[]
│   │       └── finalReveal
├── store.js
│   ├── puzzlesData                        imported from data.js
│   ├── progressByPuzzleId                one shared progress object
│   │   └── [puzzleId]
│   │       ├── currentStageIndex
│   │       └── solved
```



Setup and data

- Keep JavaScript ES modules, existing dependencies, `express.json()`, and `npm start`. Mount `puzzlesRouter` at `/api`. Register the JSON parsing error handler after the Router.
- Replace copied B4F Hub wording with project wording. Keep your existing commits.
- Use `puzzlesData.puzzles` in `data.js` and add `export { puzzlesData }`. If your branch still uses `mysteryData.mysteries`, change only those names and the export. Preserve every story, question, clue, answer, hint, stage order, and ending.
- Update the README with `npm install`, `npm start`, the port, request/response examples, and progress reset behavior. Frontend API helpers and proxy setup belong to Majed.

Store and progress

Initialize `progressByPuzzleId` once when `store.js` loads. Each puzzle starts with:

```
{ currentStateIndex: 0, solved: false, hintsUsedByStage: { 1: 0, 2: 0, 3: 0 } }
```

Build puzzle and stage keys from the seed IDs. Keep puzzle content separate from progress. All controllers import store functions; only `store.js` imports `data.js`.

- All visitors share one progress record per puzzle. Different puzzles have separate progress.
- GET requests never change or reinitialize progress.
- Wrong answers leave progress unchanged. Correct answers advance exactly one stage.
- After the final correct answer, set `solved: true` and `currentStateIndex` to the number of stages.
- Hint usage belongs to its puzzle and stage. The next hint index equals the number already revealed.
- Restarting the server resets progress. Do not add persistence or a reset API.

Endpoints and responses

Method and full path	Request body	Successful response
GET <code>/api/getAllPuzzles</code>	None	<code>{ puzzles: [...] }</code>
GET <code>/api/getPuzzle/:puzzleId</code>	None	Public puzzle detail
POST <code>/api/submitAnswer/:puzzleId</code>	<code>{ stageId: 1, answer: "3556" }</code>	<code>{ correct, message, puzzle }</code>
PATCH <code>/api/requestStageHint/:puzzleId</code>	<code>{ stageId: 1 }</code>	<code>{ hintId, hint, puzzle }</code>

All successful responses use `200`. Collection entries contain only `id`, `title`, and `summary`; return all three puzzles.

`getPublicPuzzle()` builds this detail shape from allowed fields:

```
{
  id: 1,
  title: "The Heisenberg Formula",
  summary: "...", // original seed text
  story: "...",  // original seed text
  progress: {
    currentStateId: 1,
    completedStages: 0,
    totalStages: 3,
    solved: false
  },
  currentStage: {
    id: 1,
    title: "Stage 1: The Periodic Combination",
    question: "...", // original seed text
  }
}
```

```

    clues: ["..."], // original seed strings
    revealedHints: [],
    hintsRemaining: 2
  }
}

```

Use the seed values, not the "..." placeholders. After a hint request, `revealedHints` includes `{ hintId: 0, hint: "original hint text" }`, and `hintsRemaining` decreases. Later GETs retain already revealed current-stage hints. Never return unused hints, answers, future stages, or the ending before completion.

When solved, return `currentStage: null, currentStageId: null, completedStages` equal to `totalStages`, and `solved: true`. Add `finalReveal` with the original ending string. Before completion, omit `finalReveal`. Do not access a stage beyond the array.

POST compares `answer.trim().toLowerCase()` against each accepted answer normalized the same way. Keep internal spaces and punctuation unchanged. Keep PINs as strings. Wrong answers return `correct: false`, useful feedback, and unchanged public detail. Correct answers return `correct: true`, useful feedback, and the next stage or completed detail in `puzzle`.

PATCH returns exactly the next unused hint. Its `hintId` is the zero-based index in that stage's hints array. Increase usage once and return updated detail in `puzzle`. The client sends `stageId`, not a chosen hint ID.

Validation and errors

Return errors as `{ message: "Clear explanation" }`.

Status	Cases
400	Malformed puzzle ID; missing/null/array body; invalid or missing fields; empty answer; malformed JSON
404	Valid puzzle ID does not exist; valid stage ID does not exist inside that puzzle
409	Existing stage is not current; puzzle is already solved on an answer/hint request; current stage has no hints remaining

- Validate path IDs as positive integer strings before converting. Reject `abc`, `1abc`, `0`, and `1.5`. Do not use `parseInt()` alone.
- Mutation bodies must be non-null objects, not arrays. `stageId` must be a positive integer number. POST `answer` must be a string that is not empty after trimming.
- Use plain `if`, `typeof`, `Array.isArray`, and `Number.isInteger` checks. Check puzzle/stage existence before returning a state conflict.
- Rejected requests never change progress. Wrong answers are `200` game feedback.
- Keep validation and mutation synchronous; do not place an `await` between them.
- `handleJsonParseError` handles Express JSON parsing failures with `400` and the agreed message shape. Pass other errors to `next(err)`.

Rules

- Use the taught Express entry point, Router, controller, and store structure. No extra service/repository layers, dependency injection, or complex error middleware.
- No databases/ORMs, authentication/sessions/JWT, Redis, WebSockets, GraphQL, NestJS, Next.js, Docker, deployment, validation libraries, Redux async tools, or query libraries.
- Do not change puzzle content, add fake success, or add timer/score features. Build public responses explicitly; never send raw seed objects.
- Work only on your assigned branch. Do not force-push or merge into `main` yourself.
- Do not commit `node_modules`, `.env`, private notes, or private documents.

Test it

Use curl, Postman, or Thunder Client. Record requests, statuses, and responses.

1. Fresh checkout: run `npm install` in `server/`, then `npm start`. Confirm startup and run `node --check` on every changed JavaScript file.
2. GET collection: expect `200`, three summaries, and no private stage content. GET each puzzle: expect its original story and stage 1, zero completed stages, empty revealed hints, and no ending. Repeated GETs must not change progress.
3. GET IDs `abc`, `1abc`, `0`, and `1.5`: expect `400`. GET puzzle `999`: expect `404`.
4. POST puzzle 1, stage 1, answer `"wrong"`: expect `200`, `correct: false`, unchanged stage and hints. POST `" 3556 "`: expect `correct: true` and stage 2. Resubmit stage 1: expect `409`, no advancement. `"HEISENBERG"` passes stage 2.
5. On a fresh stage, PATCH twice: expect hint IDs `0`, then `1`, with exact hint text. Third request: `409`, no change. GET retains revealed hints. A new stage starts with its own unused hints.
6. Test missing/null/array bodies, missing fields, string/fractional `stageId`, number/empty/whitespace-only answers, and malformed JSON: expect `400` and no changes. Valid unknown puzzle/stage: `404`. Existing future stage: `409`.
7. Complete all three puzzles using the reference answers. Final POST and later GET return solved progress, null current stage, and the exact ending. Further POST/PATCH requests return `409`. Actions on one puzzle must not affect the others.
8. Restart the server: every puzzle returns to stage 1 with unused hints. Check all responses for leaked answers, unused hints, future stages, or premature endings.

Done when

- All four real endpoints and their failure cases pass independent checks.
- All three puzzles are playable through the APIs using one store.
- Fresh installation/start works and the README matches the implementation.
- You can explain the request flow, validation, answer matching, and hint indices.
- Majed has reviewed and merged the work.

Working time

Estimate: 3–4 focused hours. First checkpoint: startup, store, and both GET endpoints. Second checkpoint: answer/hint endpoints and full-flow checks. Raise blockers after 20–30 minutes.

Submitting

Commit meaningful steps and push to `member-eyad`. Open a PR to `main` and request Majed's review. Include endpoint test evidence and known limitations. Majed integrates the accepted backend with the frontend.